Swinburne University of Technology
School of Science, Computing, and Emerging Technologies

SWE40006 - Software Deployment and Evolution

Deployment Portfolio Task 3

Student name: Ngoc Van Nhi Do
Student ID: 105551765
Submission date: DON'T FORGET THISSSSSSSSSSS

Deployment Portfolio Task 3

1 Declared Target Level

All four subtasks 3.1, 3.2, 3.3, and 3.4 were attempted.

Table 1 below contain relevant links as proof of successful task execution.

Table 1: Publicly accessible URLs for live grading verification

Source	URL
Task 3.1	http://13.55.158.77

2 Execution Evidence

2.1 Task 3.1

For this task, I deployed a WordPress application onto an Amazon EC2 instance.

2.1.1 AWS Environment Setup

First, I created a new AWS account on the free plan, which gave me $100 in free credits.

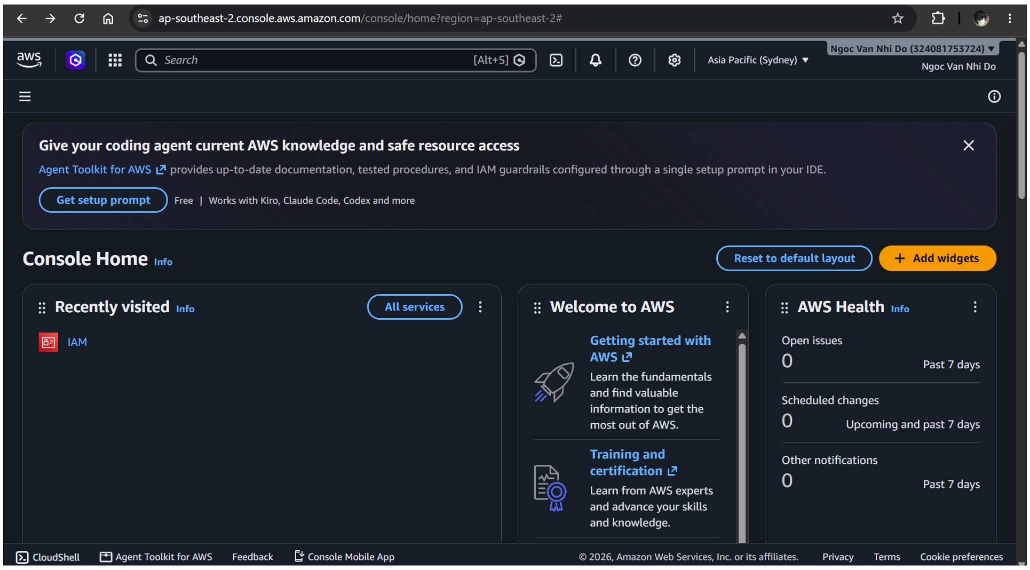

Figure 1: The AWS Console interface after I had successfully signed up.

I then created a new IAM user for my coursework named `SWE40006_student` rather than using the root account. This user is assigned to a new user group `EC2-Lab-Users` which is given full access to EC2 resources (using `AmazonEC2FullAccess`).

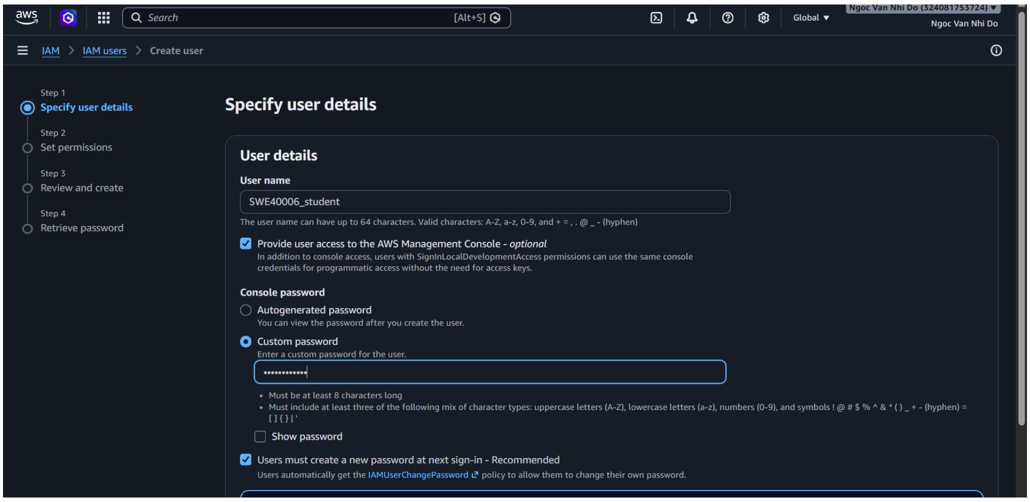

Figure 2: Creating a new IAM user SWE40006_student.

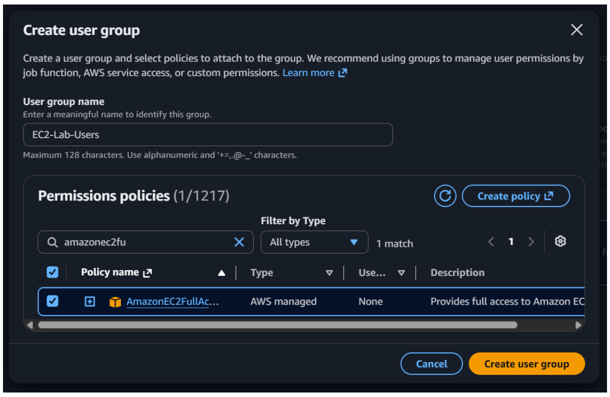

Figure 3: Adding AmazonEC2FullAccess to the user group EC2-Lab-Users, which is then assigned to SWE40006_student.

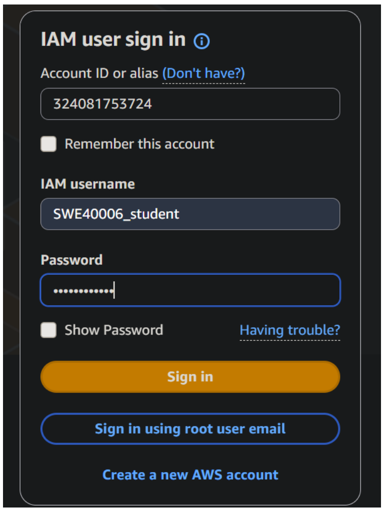

Figure 4: Logging into the AWS Console as the new IAM user.

To securely login into my EC2 instances, I must generate an SSH key pair. The private key was downloaded and stored on my local machine for later authentication.

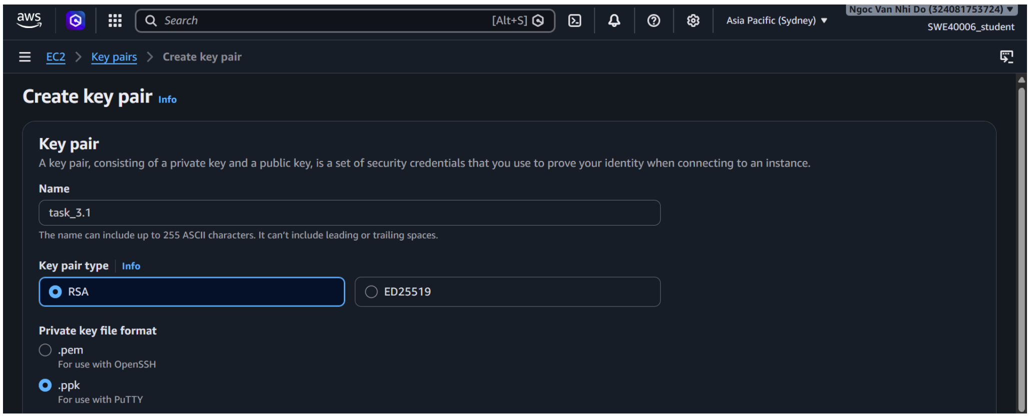

Figure 5: Creating a new key pair `task-3.1`.

2.1.2 Compute Provisioning

I launched a new EC2 instance called `Task 3.1`. This instance runs Amazon Linux 2023, uses a `t3.micro` instance time to remain eligible for the free tier, and is configured with the previously generated key pair.

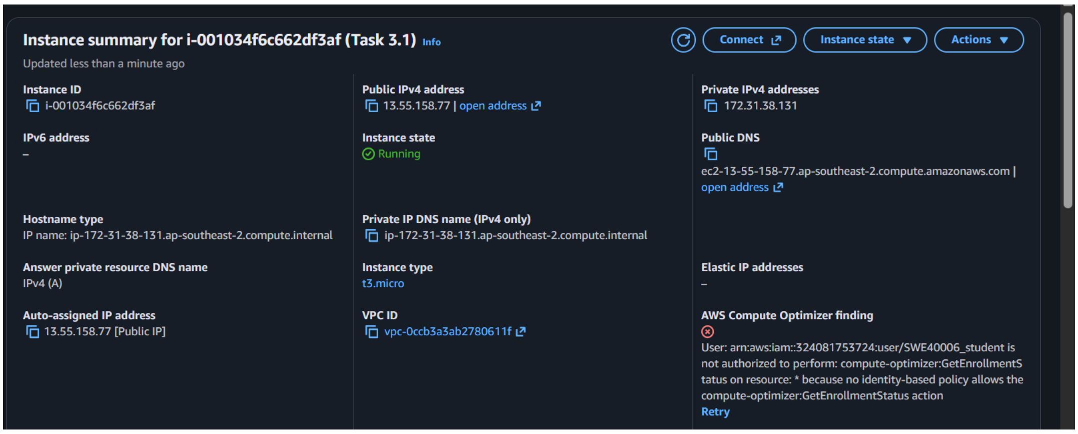

Figure 6: Creating a new Amazon EC2 instance.

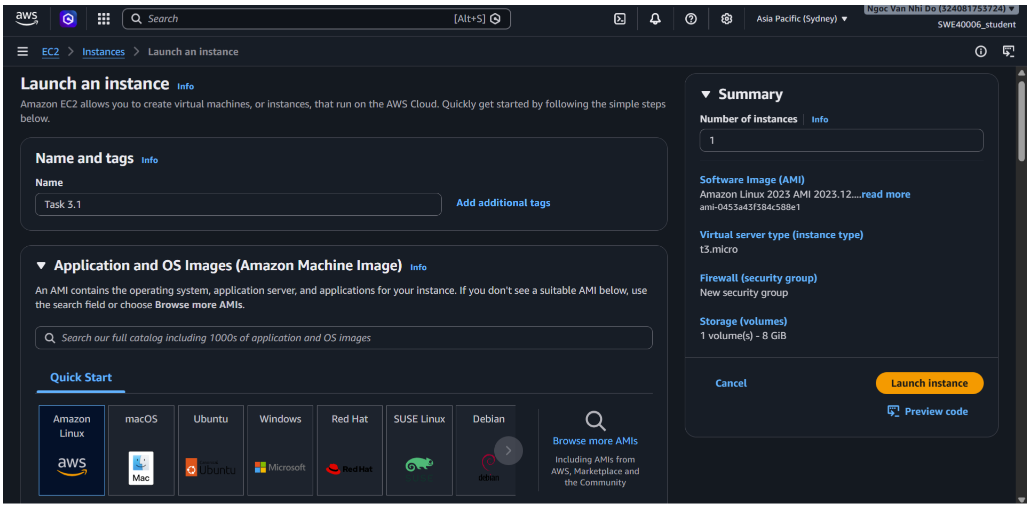

Figure 7: Amazon Linux 2023 was chosen as the OS for this instance.

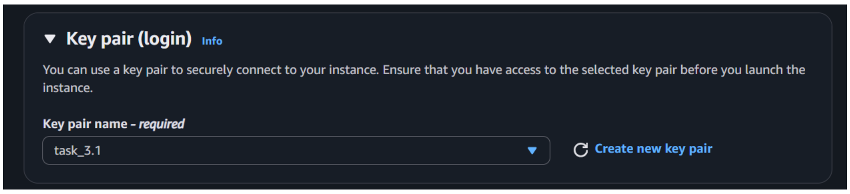

Figure 8: Configuring the instance with the `task-3.1` key pair.

I then configured the instance with a new security group `task-3.1-sg`, which allowed SSH (22), HTTP (80), and HTTPS (443) inbound traffic from anywhere.

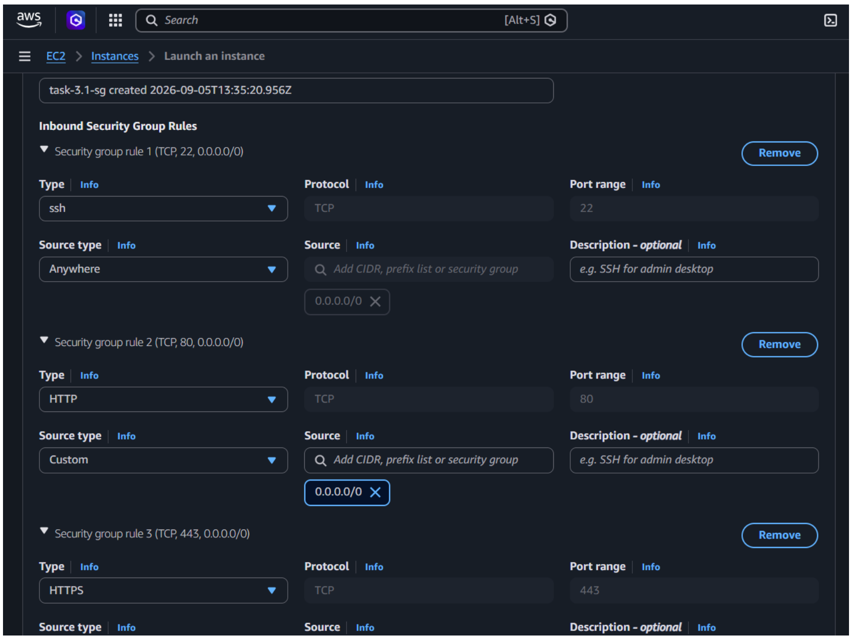

Figure 9: Configuring the security rules to allow inbound SSH, HTTP, and HTTPS traffic to this instance.

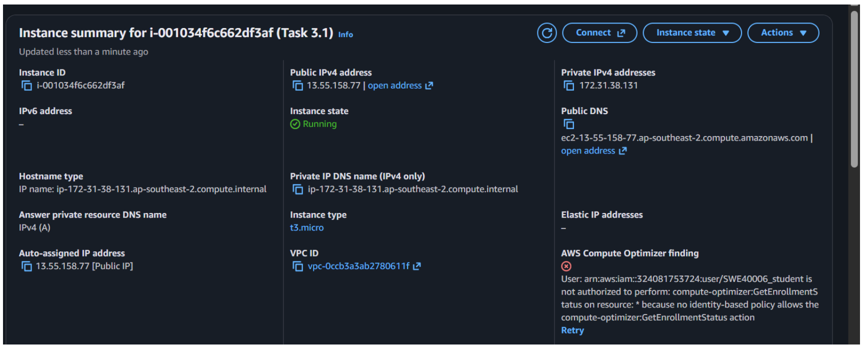

Figure 10: The new instance is successfully created.

I then SSH-ed to the instance using PuTTY.

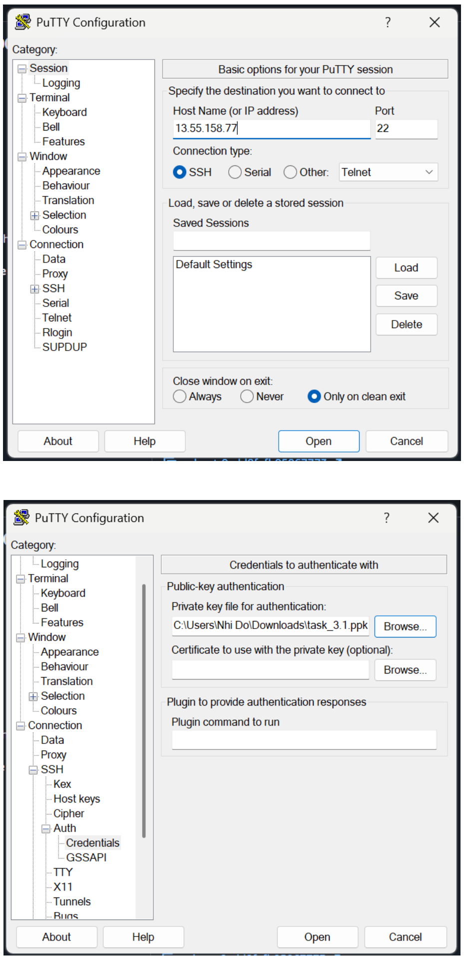

Figure 11: Connecting to the EC2 instance using SSH.

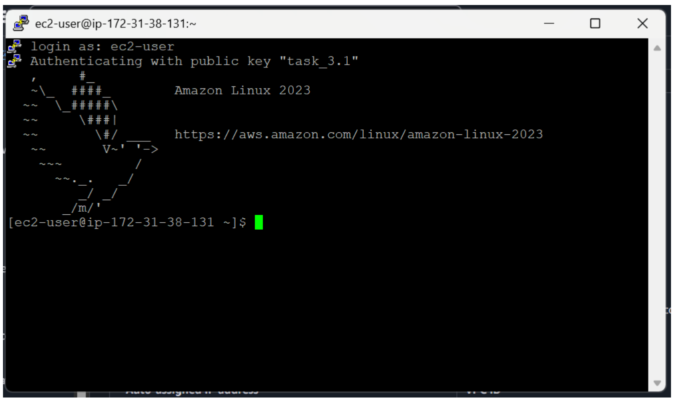

Figure 12: Connection was successful.

I then installed the Apache web server and started it so that it could later host my web application.

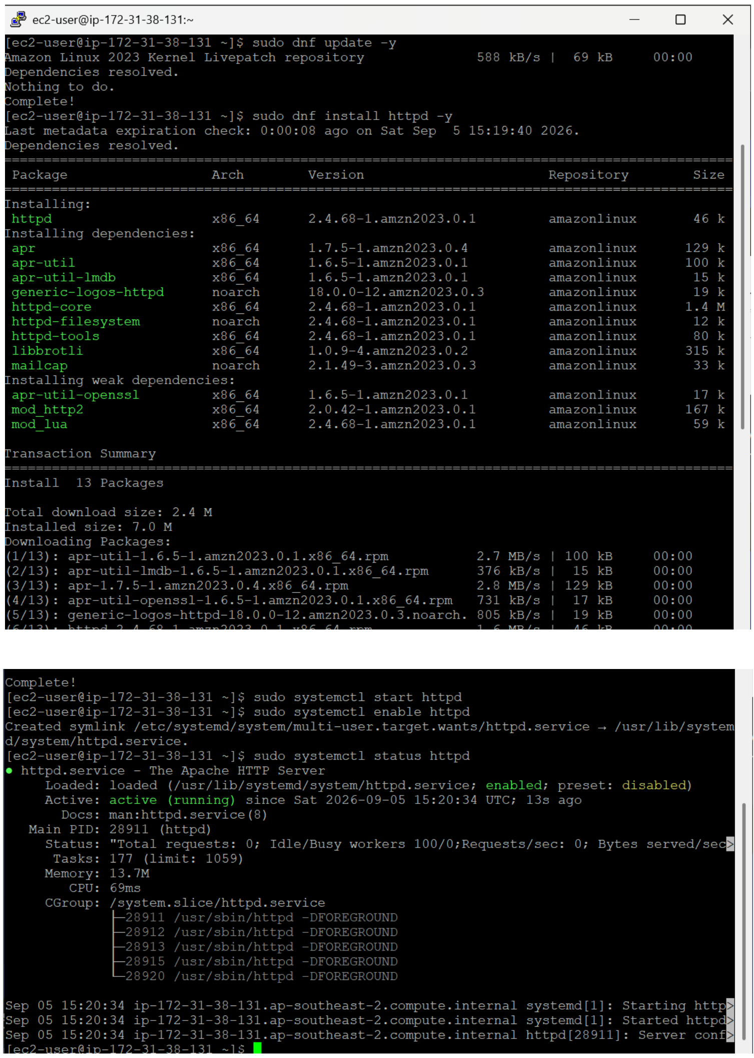

Figure 13: Installing and starting the web server.

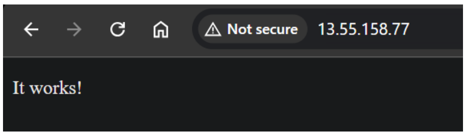

Figure 14: Verifying that the web server is publicly accessible.

2.1.3 Application Deployment

Before deploying the WordPress application, I needed to install its dependencies which included installing PHP and MariaDB.

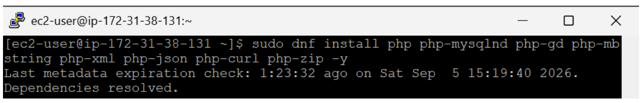

Figure 15: Installing PHP.

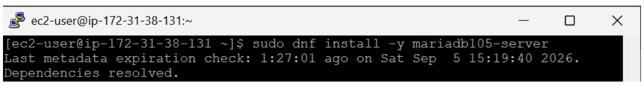

Figure 16: Installing MariaDB.

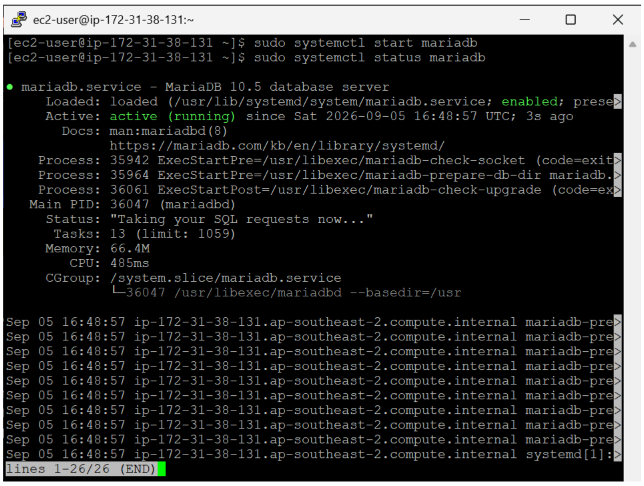

Figure 17: Starting and enabling the MariaDB service.

I then set up a local database for the web application.

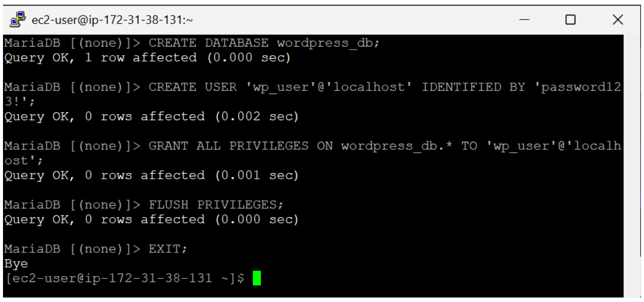

Figure 18: Setting up a local database named `wordpress_db` for the WordPress application.

I installed and extracted the WordPress package. The command `chown` made Apache the owner of the WordPress files, while `chmod` set who could read, write, or access those files. These commands ensured that Apache had the correct permissions to run WordPress.

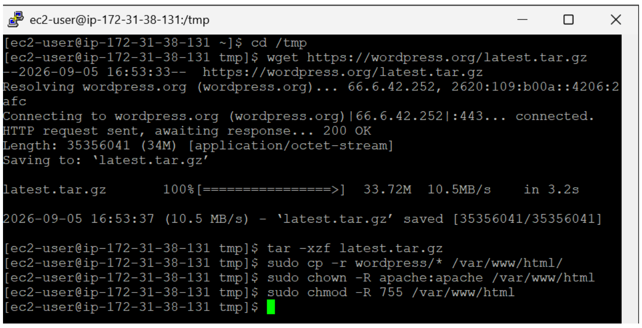

Figure 19: Installing WordPress and configuring permissions.

My WordPress application is deployed and running at http://13.55.158.77.

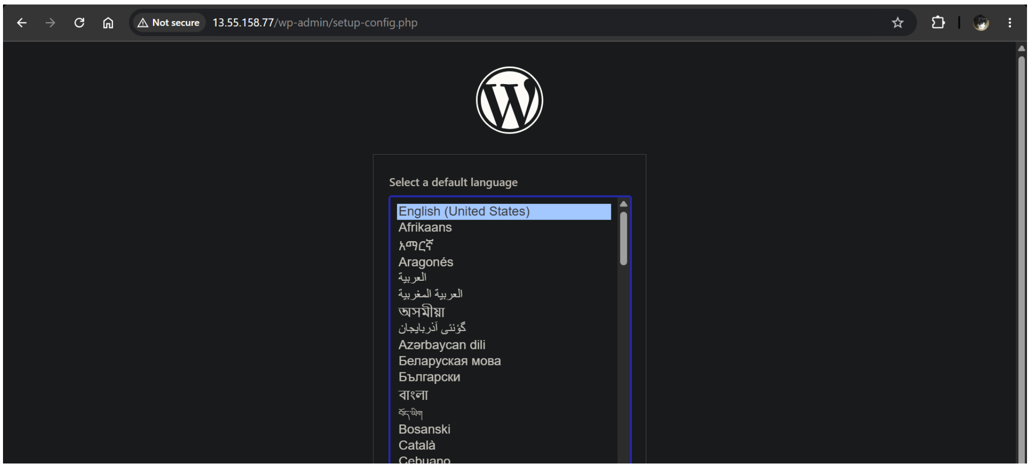

Figure 20: The deployed WordPress application.